

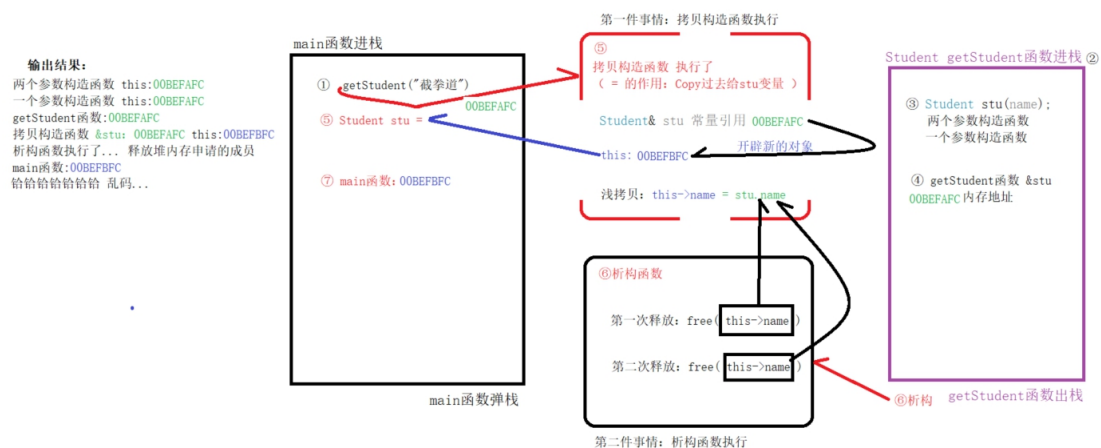
C++浅拷贝与深拷贝的原理（NDK 第九节课）

旧地址 新地址：

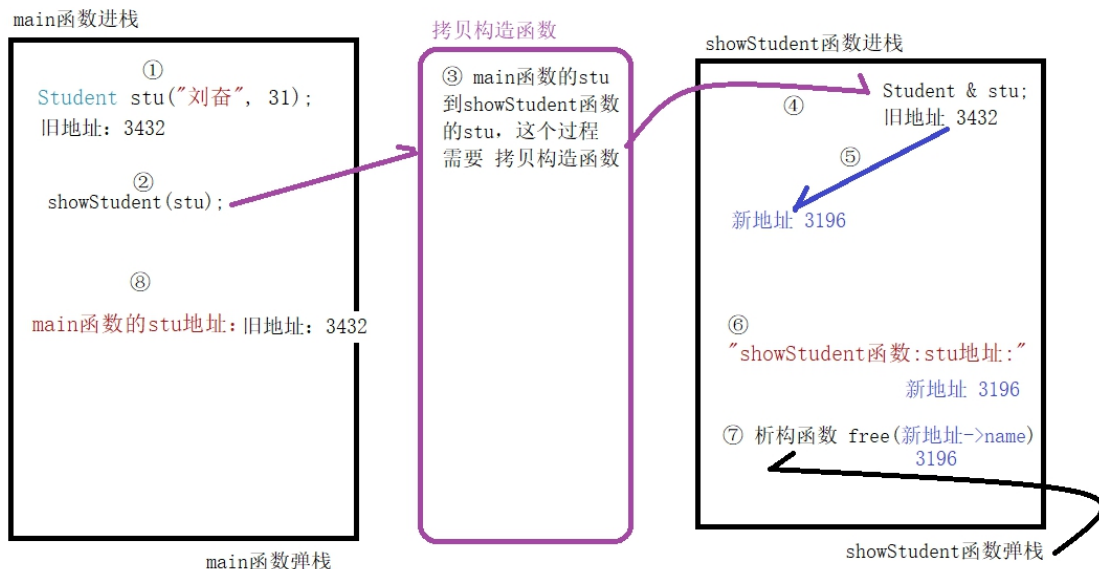
```
Student(const Student & stu) {  
    stu 就是旧地址  
  
    this 就是新地址  
}
```

默认 隐士的 拷贝构造函数 我们看不见
如果我们自定义了拷贝构造函数，就会覆盖默认的拷贝构造函数
和Java的构造函数是一样的思路

拷贝构造函数流程图：



拷贝构造函数执行图 2：



报错总结:

案例一, 为什么会报错? 纠结

```
void main() {
    Student stu=getStudent("截拳道");
}
```

释放的是 拷贝构造函数 新地址,
为什么也会重复释放?
之前重复释放的是 旧地址 我们一直认为是重复释放旧地址导致的问题
现在只是释放了一次新地址 也报错? 到底是什么原因? 娓娓道来

案例二, 为什么也报错, 纠结

```
void main() {
    Student stu("刘奋", 31);
    showStudent(stu);
    showStudent(stu);
}
```

多调用一次此函数, 就报错, 两次释放新地址而已, 到底是什么原因? 娓娓道来

案例一 案例二 报错总结:

1. 释放两次旧地址 会报错。
2. 释放一次旧地址, 一次新地址, 也报错。
3. 释放两次新地址, 也报错。

C++浅拷贝与深拷贝的原理 (NDK 第九节课)

20: 05 准时开始上课 (以后一直是这个时间)

讲师简介



Derry

9年项目研发经验，曾就职于腾讯IOT项目组、华为安防研发组、中国联通担任资深工程师

研究领域：Android操作系统定制，JavaEE服务器，前端，RN跨平台，参与框架设计/腾讯物联SDK研发/华为平台SDK技术研发，NDK领域与音视频领域等等

开发环境：VS2013 专门学习 拷贝构造函数

C++浅拷贝与深拷贝的原理（NDK 第九节课）

1. 拷贝构造函数与析构函数执行流程分析
2. 拷贝构造函数与析构函数原理细节图研究
3. 拷贝构造函数配合析构函数制作崩溃
4. 深拷贝解决崩溃，并分析原理

预习资料：暂无预习资料，重点是要把前面的内容补上来

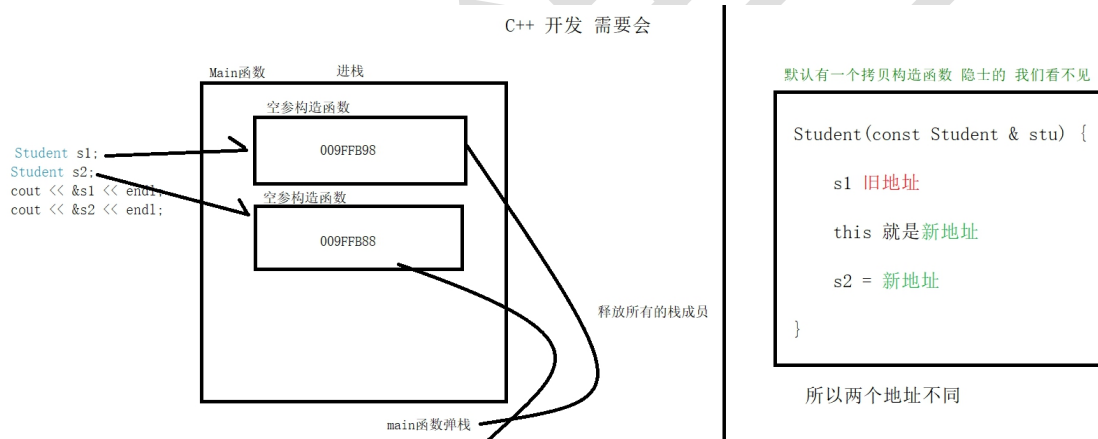
21: 10 准时开始上课

上课时的临时草稿画图记录:

二个参数构造函数 this:004FF938 旧地址
一个参数构造函数 this:004FF938
getStudent函数:004FF938
拷贝构造函数 &stu:004FF938 this:004FFA38
析构函数执行
main函数:004FFA38 新地址

浅拷贝: 会报错, 析构函数执行 重复释放 &this->name:21409728
二个参数构造函数 this:17823856
拷贝构造函数 &stu:17823856 this:17823620 新地址 拷贝构造函数完成
showStudent函数:17823620 刘奋, 31
析构函数执行 &this->name:21409728

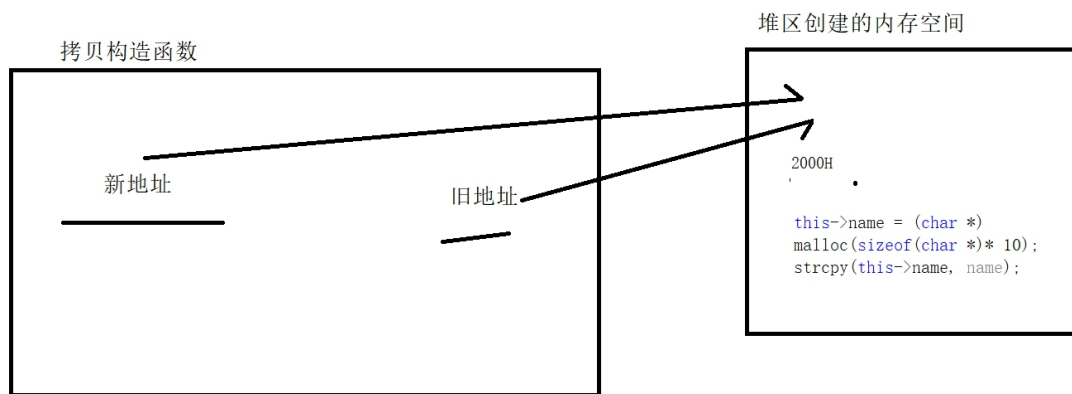
旧地址, 新地址, T1 的分析:



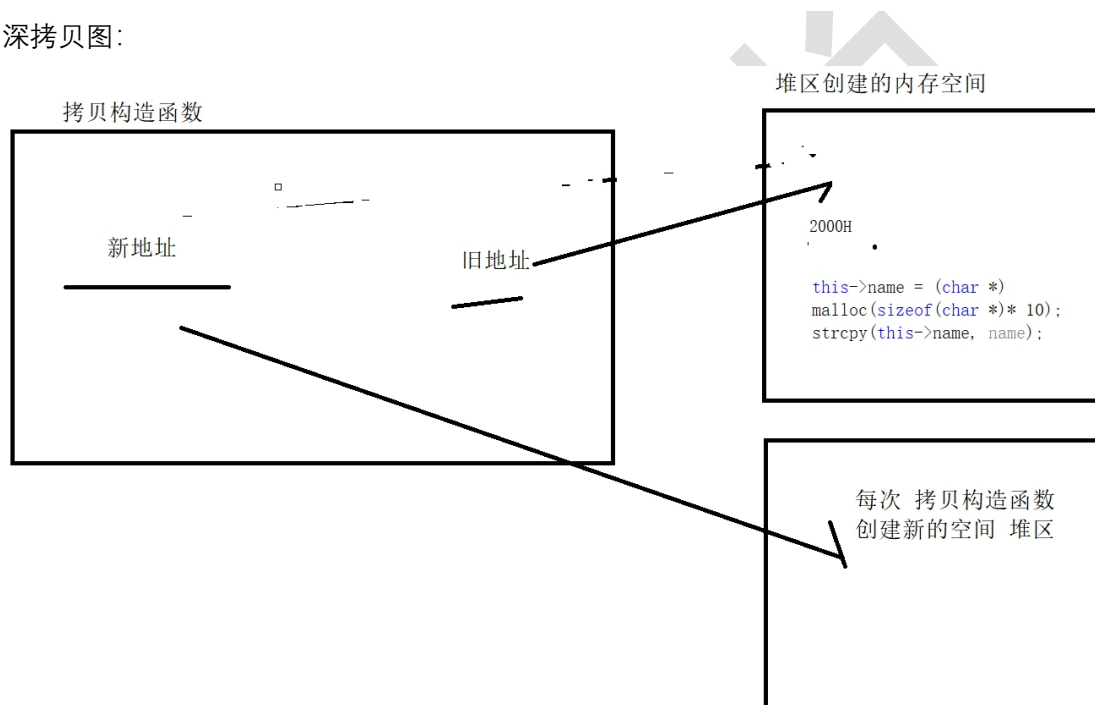
拷贝构造函数执行流程, T2 的分析:



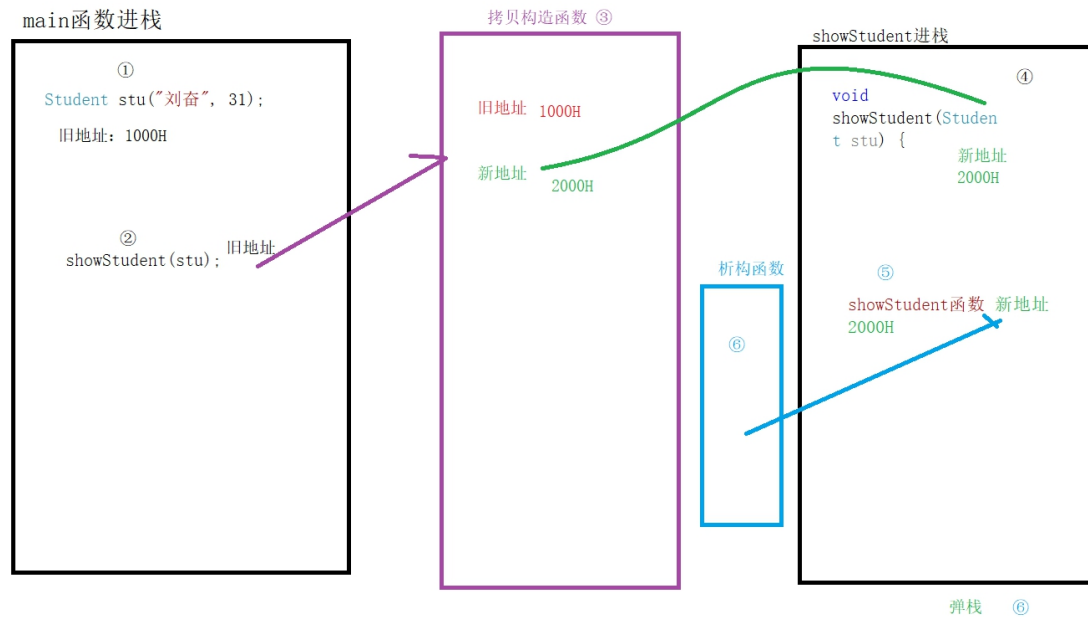
浅拷贝带来的问题：



深拷贝图：



拷贝构造函数 T3 的分析：



课堂上的草稿:

二个参数构造函数 this:004FF938
 一个参数构造函数 this:004FF938
 getStudent 函数:004FF938
 拷贝构造函数 &stu:004FF938 this:004FFA38
 析构函数执行 --- getStudent 函数弹栈
 main 函数:004FFA38

浅拷贝: 会报错, 析构函数执行 重复释放 &this->name:21409728

二个参数构造函数 this:17823856
 拷贝构造函数 &stu:17823856 this: 17823620
 showStudent 函数:17823620 刘奋,31
 析构函数执行 &this->name:21409728 新地址的 name

二个参数构造函数 this:13892892
 拷贝构造函数 &stu:13892892 this:13892656
 拷贝构造函数 2 this->name:15642632 stu.name:15642632
 showStudent 函数: 13892656 刘奋,31
 析构函数执行 &this->name:15642632

进入答疑环节, 没有问题同学, 可以睡觉了